The Grammar of Graphics Needs Verbs: Towards a GUGOG

Josh M. Pollock[1]
Independent Researcher, Somerville, MA, USA

Abstract

The Grammar of Graphics (GoG) made significant strides toward unifying previously incompatible chart abstractions into a shared conceptual model. Today, we face a related problem. Inspired by the GoG, we have a proliferation of domain-specific grammars whose compositional ideas are difficult to compare and combine.

I argue that one source of this divergence is that while the GoG gives us a powerful set of nouns for statistical graphics (marks), it provides no equally general set of verbs for composing marks together. Through this lens, we can understand recent grammars as specialized extensions of the GoG with verbs for arranging, compositing, repeating, or otherwise relating marks to each other. But because each grammar introduces these operations in its own language, closely related ideas are still difficult to compare, combine, and carry into practice.

In this position paper, I show how the GoFish grammar's relations and operators can represent three recent grammars in a shared system. GoFish allows the same verbs to be used across all three systems, making it easier to compare and contrast grammars and identify their core contributions.

Index Terms: Visualization grammars, Grammar of graphics, Visualization systems, Programming languages

1 Introduction

The Grammar of Graphics (GoG) replaced inconsistent collections of chart types with a consistent set of seven concepts that could be independently composed with each other [14]. This language is extremely expressive. It can represent most common chart types. It's also able to support a vibrant ecosystem of custom marks in ggplot2 [2]. Not only is it expressive, but it also provides a simple, clarifying formal model of how charts encode information. Nevertheless, the GoG does not cover every visualization and researchers continue to create new grammars for trees, networks, maps, animation, interaction, scientific figures, and other domains [7].

We now find ourselves in a world, not of chart types, but of *grammar types*. Influential systems such as Atom, Mascot, and GoTree [4–6, 8] are built as end-to-end systems, from surface specification to layout to rendering, often on conceptual foundations that only roughly mirror the GoG's. These separate foundations make it difficult to compare concepts across research systems and build on prior work. And because they do not build directly on the GoG, it's difficult to translate research insights to production visualization systems like Matplotlib or ggplot2.

I claim that a central cause of this cacophony is the relative lack of *verbs* in the GoG. While the GoG has many "nouns" (i.e., marks) and "adjectives" (i.e., encoding channels), the language lacks "verbs" that take graphical objects and compose them into new graphical objects. While graphics can be composed with facets, coordinate transformations, and position adjustments, these primitives do not express all the constructions needed by specialized grammars. I believe that much of the drift away from the GoG in specialized visualization grammars is spurred by its lack of graphical verbs.

In this position paper, I show how the GoFish grammar's relations and operators [11] can represent three recent grammars in a shared system. GoFish allows the same verbs to be used across all three systems, making it easier to compare and contrast grammars and identify their core contributions.

2 Background: GoFish Composition

GoFish extends the GoG with relations and operators for composing marks together. We give a brief overview of and motivation for these abstractions. For additional details, see the Bluefish and GoFish papers [10, 11] and `gofish.graphics`.

Diagrams and charts communicate not only through the properties of individual marks, but also through relations between marks. Larkin and Simon describe diagrams as relational structures that support perceptual inference [3]. Gestalt grouping principles provide a vocabulary for these relations. Figure 1 shows four examples of Gestalt principles: common region, connectedness, alignment, and proximity. These principles affect which marks readers perceive as belonging together [13]. Pinker similarly argues that readers construct visual descriptions from marks, encodings, and Gestalt principles [9].

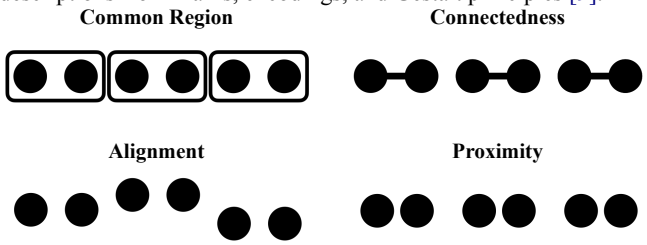


Figure 1: Four Gestalt grouping principles.

GoFish turns these theories of graphical perception into a formal language for composing graphics. We do so with two main constructs: relations and operators.

2.1 Relations

GoFish **relations** relate two or more marks together. Relations can be spatial constraints (such as alignment) or marks themselves (such as connectedness).

2.1.1 Constraints

Two or more elements can be spatially related with *constraints*. For example, to distribute elements A and B 20 pixels apart, a user can write `Constraint.distribute({dir: "x", spacing: 20}, [A, B])`. This constraint is placed in a `.relate()` clause on a `layer`:

```
layer([rect(...).name("A"), circle(...).name("B")])
  .relate(({ A, B }) => [
    Constraint.distribute({dir: "x", spacing: 20}, [A,
B])])
```

The `layer` overlays elements on top of each other. The `.name()` modifier provides a name for an element so that it can be related in `.relate()`.

2.1.2 Relational Marks

To draw a line between the two marks, we can use a *relational mark*. Like a typical mark, a relational mark can render something on the screen. However, unlike a typical mark, a relational mark draws itself relative to its children.

The spec from before can be extended with a relational mark, `ribbon`, that draws a ribbon between its children (in this case connecting the children horizontally):

```
layer([rect(...).name("A"), circle(...).name("B")])
  .relate(({ A, B }) => [
    Constraint.distribute({dir: "x", spacing: 20}, [A,
B]),
    ribbon({dir: "x"}, [A, B]),
  ])
```

[1]joshpollock1997@gmail.com. Work begun while affiliated with MIT.

2.1.3 Composite Relations

Primitive relations like `distribute` and `ribbon` are very expressive, but there are often times when two or more relations appear frequently together. Using the `layer` + `.relate()` machinery above, we can create reusable composite relations. For example, the `spread` relation aligns its children on one axis and distributes them on another. This is useful for creating lists of marks or spreading rectangles to make a bar chart. Composite relations can be called in `.relate()` like primitive relations, but if they also ensure their children are fully laid out, they can be called outside of a `.relate()` method. For example, a user may write `spread({dir: "x", spacing: 20}, [rect(...), circle(...)])`, similar to a component in a user interface framework. These direct calls allow users to construct hierarchical specifications without the verbose, but more expressive, `layer` + `.relate()` syntax.

2.2 Operators

GoFish relations are useful to relate individual marks together, but in charts we often want to relate arrays of data-driven marks simultaneously. GoFish provides an **operator** syntax for relations to support this. Unlike more general relations, operators must ensure their children are fully laid out. That means that `spread` can be an operator since it fully specifies its children's layout, `ribbon` can be an operator since it requires its children to already be placed, but `distribute` cannot be, since it only constrains one axis of its children.

Figure 2 shows a ribbon chart written with operators. `chart` provides the `seafood` dataset. `spread` groups the data by lake and spreads each of those regions out. Inside each lake, `stack` groups its data by species, orders the species by count, and vertically stacks them without any spacing. `rect` is evaluated once per resulting lake/species scope. The `height` and `fill` channels refer to fields in `seafood` similar to Observable Plot's syntax [1]. Finally, the `.layer()` method adds a second layer with a `ribbon` relational mark. This mark connects the marks from the previous layer with ribbons.

2.3 Relationship to the GoG

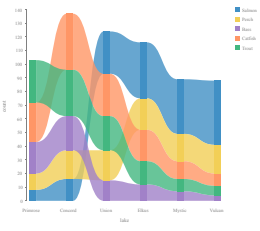
Familiar GoG marks can be recovered as compositions of GoFish operators and marks. For example, in Observable Plot we can write a bar chart as

```
barY(alphabet, { x: "letter", y: "frequency" })
```

In GoFish this corresponds to the composition of the `spread` operator and `rect` mark:

```
chart(alphabet)
  .flow(spread({ by: "letter", dir: "x" }))
  .mark(rect({ h: "frequency" }))
```

As with the GoG, GoFish automatically infers scales, axes, and legends from the specification, and a user can transform a chart using a coordinate transformation. An end-user can write a custom `barY` mark to wrap this lower-level GoFish code that can be used like `rect`.



```
chart(seafood, { axes: true })
  .flow(
    spread({
      by: "lake", dir: "x",
      spacing: 64
    }),
    stack({
      by: field("species")
        .sort("count"),
      dir: "y"
    })
  )
  .mark(rect({
    h: "count", fill: "species"
  }))
  .layer(ribbon({
    by: "species", opacity: 0.8
  }));
```

Figure 2: A GoFish ribbon chart: operators arrange the rectangles, and a relational mark connects them.

Unlike in the GoG, charts can be nested in other charts in GoFish. For example, suppose we want to analyze the alphabet frequencies across many languages. We might write

```
chart(languages)
  .flow(spread({ by: "language", dir: "x" }))
  .mark(chart()
    .flow(spread({ by: "letter", dir: "x" }))
    .mark(rect({ h: "frequency" })))
```

Notice that GoFish's `spread` operator has subsumed the role of GoG faceting. Just like `charts`, `layers` can also be used inside a `.mark()` call to create glyphs. This nesting behavior provides a different kind of composition, *recursive* composition, than the GoG's flat combinatorial composition, allowing GoFish to support more kinds of charts.

3 Specialized Grammars in a Shared Vocabulary

Recently developed specialized grammars provide seemingly incompatible abstractions. Now that we are armed with GoFish's compositional structures, we can revisit these grammars and see that they are not so different after all. In this section we show how Atom [8], PiCCL [12], and GoTree [4] can be expressed using relations and operators.

3.1 Atom

Atom [8] is a grammar for unit visualizations. Its recursive pipeline syntax was an inspiration for GoFish's operator syntax. In Figure 3, Atom's Titanic example first groups passengers by class, then lays out the passengers within each class as circles colored by survival. GoFish expresses the same construction in one `flow`: a `spread` operator groups classes, data transforms order and chunk each class's passengers, and two more `spread` operators arrange the rows and columns. A unit layout is thus a composition of data transformations, graphical operators, and a mark.

3.2 PiCCL and Mascot

Mascot and PiCCL provide composition operations inspired by graphics editors [5, 12]. Figure 4 places the bottle specification from the PiCCL website beside its GoFish counterpart. Both compose an image with a data-sized rectangle, attach a line and percentage label, and repeat the resulting glyph for four categories. Though PiCCL and GoFish are inspired by different approaches, direct manipulation editors and declarative GoG languages, respectively, they have independently converged on very similar abstractions. This strongly suggests that relations and operators are useful, general abstractions. Where PiCCL and GoFish differ in this example is mainly in how data-driven values are described (`.mapValue()` vs polymorphic encoding channels) and how operators are defined (`replicate` and layouts vs operators that also group data by a specific key).

3.3 GoTree

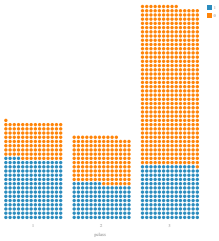
GoTree [4] specifies unit tree visualizations by decomposing parent-child and sibling relationships into horizontal and vertical rules. In its node-link example (Figure 5), parents are centered above their descendants, while sibling subtrees are distributed and aligned at their tops. Unlike Atom and PiCCL, which are based on array data structures, GoTree is based on hierarchical data. This poses a different kind of challenge for a GUGOG than the previous two systems.

We designed GoFish's operator syntax to work well with arrays, and it is not well-suited to trees. Fortunately, because we built operators on the lower-level relational syntax, we can build a higher-level syntax for tree-like structures that elaborates into relations like operators do. To that end, we implemented the `gofish-gotree` library on top of GoFish relations. By building on top of GoFish we're able to reuse the same `align` and `distribute` relations and the `line` mark in a GoTree-like language, and we get support for coordinate transforms, scales, and custom marks without having to design them from scratch. The need for this additional design suggests there could be syntaxes well-suited to other data structures that could be built

Atom

```
{
  "layouts": [{
A   "type": "gridxy",
    "subgroup": {"type": "groupby", "key": "pclass"},
    "aspect_ratio": "fillX",
    "size": { "type": "uniform", "isShared": false },
    "direction": "LRBT", "align": "LB"
  }, {
B   "type": "gridxy",
    "subgroup": { "type": "flatten" },
    "aspect ratio": "maxfill",
    "size": { "type": "uniform", "isShared": true },
    "direction": "LRBT", "align": "LB",
    "sort": { "key": "survived" }
  }],
C "mark": { "shape": "circle",
    "color": {"key":"survived", "type":"categorical"},
    "size": { "type": "max", "isShared": false },
    "isColorScaleShared": true }
}
```

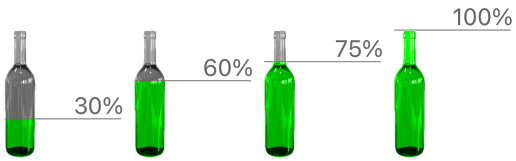
GoFish



```
chart(titanicPassengers)
  .flow(
A   spread({ by: "pclass", dir: "x",
      spacing: 24, alignment: "end" }),
B   derive(rows => orderBy(rows,
      ["survived"], ["desc"])),
    derive(rows => chunk(rows, 14)),
    spread({ dir: "y", spacing: 2, reverse: true }),
    spread({ dir: "x", spacing: 2 })
  )
C .mark(circle({ r: 4, fill: "survived" }));
```

Figure 3: GoFish rendering of `Atom's Titanic example`. Blue dots indicate survival. Setup is omitted. Atom packs units automatically; GoFish uses a fixed 14-column grid.

on GoFish relations as well, such as for networks, cartography, or genomics.



PiCCL

```
const scale = new PICCL.LinearScale({range: [0,400]});
A const pic = new PICCL.Picture({
    url: pictureUrl, height: 400, color: "#fff"
  });
  const colorFill = new PICCL.Rect({
    height: 400, fill: "#00ff00"
  }).mapValue("height", "amount", { scale });
  const coloredPic = PICCL.atop(
    pic, colorFill, "luminosity")
    .lengthMatch(colorFill, pic, { channel: "width" });
B const line = new PICCL.Line({
    x1: 0, y1: 0, x2: 250, y2: 0
  }).mapValue("y", "amount", { scale });
C const label = new PICCL.Text({
    fontSize: 36, fill: "#666",
    textAnchor: "end", suffix: "%"
  }).mapValue("content", "amount");
D const glyph = PICCL.union([coloredPic, line, label])
E   .pointSnap(label, line, { anchor: [1, 0] });
F const collection = PICCL.replicate(glyph)
    .stackLayout("x", 10);
  await PICCL.show([[collection, data]],
    container, { pictures });
```

GoFish

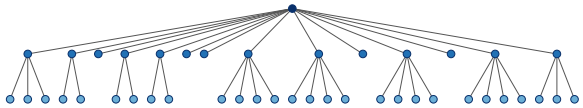
```
A const pic = image({ href: bottlePng, h: v(100) });
  const colorFill = rect({ h: "amount", w: 175,
    fill: "#00ff00" });
  const coloredPic = paint({ blendMode: "color" },
    [pic, colorFill]).name("bottle");
B const line = rect({ h: 1, w: 175,
    y: "amount", fill: "#666" }).name("line");
C const label = text({ text: d => `${d.amount}%`,
    fontSize: 35, fill: "#666" }).name("label");
D const glyph = layer([coloredPic, line, label])
E   .relate(({ bottle, line, label }) => [
      Constraint.align({ x: "start" }, [bottle, line]),
      Constraint.distribute({ dir: "y", spacing: 0 },
        [line, label]),
      Constraint.align({ x: "end" }, [label, line]),
    ]);
F const collection = chart(data, { axes: false })
    .flow(spread({ by: "category", dir: "x",
      spacing: 20 }))
    .mark(glyph);
```

Figure 4: GoFish bottle rendering with `PiCCL's source` at left. Data, imports, and asset setup are omitted; image sizing and blending differ.

GoTree

```
A  "Element": { "Node": "circle",
B                "Link": "straight" },
   "CoordinateSystem": { "Category": "cartesian" },
   "Layout": {
     "Category":"AxisIndependent", "Mode":"bottom-up",
C    "X": { "Root": { "Relation": "within" },
D      "Subtree": { "Relation": "flatten",
                    "Margin": "0.3w" }
     },
E    "Y": { "Root": { "Relation": "juxtapose",
                      "Margin": "0.2" },
F      "Subtree": { "Relation": "align",
                    "Alignment": "top" }
     },
     "SubtreeWidth": "adaptive",
     "SubtreeHeight": "adaptive"
   }
 }
```

GoFish / gofish-gotree



```
tree({
A  node: () => circle({ r: 9 }),
B  link: { kind: "line", curve: "linear" },
   parentChild: combine({
C    x: { kind: "align", alignment: "middle" },
E    y: { kind: "distribute", spacing: 90 },
   }),
   sibling: combine({
D    x: { kind: "distribute", spacing: 24 },
F    y: { kind: "align", alignment: "start" },
   }),
}, hierarchy);
```

Figure 5: GoFish rendering of `GoTree's node-link example`. The `link` field selects GoFish's `line` mark. Data and styling are omitted. GoTree uses relative margins and adaptive dimensions; GoFish here uses fixed gaps.

3.4 Composing Grammars

Representing existing grammars in a single language provides several benefits. One is that the relationships between language abstractions become clear. Unit visualizations, bottle charts, and trees rely on `spread` and its constituent relations, `align` and `distribute`. But when each is supported by a different language, this is hard to see. In Atom, `spread` is called `gridxy` with the `fillX` aspect ratio, while in PiCCL it's a `stackLayout`. In PiCCL, `align` is called `pointSnap`, whereas in GoTree it's called both `within` and `align`. Furthermore, once each grammar is expressible in a shared language, we can compose them to make new kinds of charts. Figure 6 composes constructs from all three grammars in a single graphic.

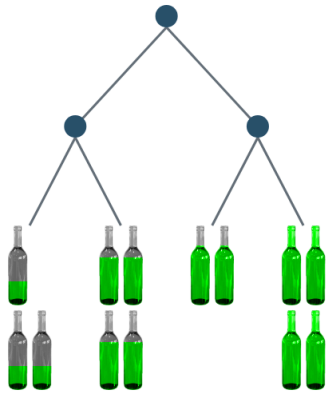
4 Conclusion and What's Next

GoFish's relations and operators extend the GoG with infrastructure for defining and using custom "verbs." These concepts allow us to represent several existing grammars in a shared language, supporting better comparison between language designs. By adding to the GoG's existing concepts rather than creating a language completely from scratch, GoFish may provide a smoother path from grammar research to production.

We are in the process of extending GoFish relations and operators to support animation, hoping to further unify animation grammars such as Animated Vega-Lite, Gemini, and Canis. We are also investigating how to make layout algorithms more compositional and extensible without sacrificing performance.

Is there one language to rule them all? Not a finite one, no. But I believe there can be a universal language of data-driven graphics that is easier to extend than the GoG, allowing us to move from a cacophony of incompatible grammars to a cumulative research programme.

In fact, I believe that the design and implementation of a GUGOG is very close indeed, perhaps a year or two away at most. Assuming a GUGOG is not so far away, what's next? The GoG is an interesting DSL from a research perspective in that it is quite hard to extend. This has made new grammar research more difficult to conduct than it could be. The GoG is difficult to extend because it is a very large language with fuzzy interface boundaries. A mark, a scale, a facet—none of these concepts are well-defined or universally agreed upon, and so it is hard to make progress on related ideas. Moreover, we rarely see small, pedagogical implementations of the GoG that could accelerate researchers' and students' understanding of the language. This is in stark contrast to other universal languages like the lambda calculus and relational algebra whose formal specifications can fit on a napkin. Small, universal designs like these are much easier to extend and modify. I think we should pursue the "essence" of data-driven graphics, a small, extensible, universal language covering not just statistical graphics, but diagrams, notations, cartography, and more.



```
const bottleUnit = data => {
  const amount = data[0]?.amount ?? data.amount;
  return paint({ blendMode: "color" }, [
    image({ href: bottlePng, h: 50 }),
    rect({ y: 50 - amount / 2,
      h: amount / 2, w: 87.5,
      fill: "#00ff00" })
  ])(data);
};

const waffle = d => chart([d.data], { axes: false })
  .flow(
    derive(rows => rows.flatMap(
      row => repeat(row, "count"))),
    derive(rows => chunk(rows, 2)),
    spread({ dir: "y", reverse: true }),
    spread({ dir: "x" }))
  .mark(bottleUnit);

tree({
  node: d => d.height ? circle({ r: 7 }) : waffle(d),
  link: (_source, target) => ({
    kind: "line", curve: "linear",
    source: { x: "middle", y: "end" },
    target: { x: "middle",
      y: target.height === 0 ? "start" : "middle" }
  }),
  parentChild: combine({
    x: { kind: "align", alignment: "middle" },
    y: { kind: "distribute", spacing: 55 }
  }),
  sibling: combine({
    x: { kind: "distribute", spacing: 30 },
    y: { kind: "align", alignment: "start" }
  })
}, hierarchy);
```

Figure 6: A GoFish graphic combining a tree, waffles, and images.

References

[1] M. Bostock and P. Rivière, "Observable Plot," 2020. https://github.com/observablehq/plot.

[2] ggplot2 Contributors, "Extending ggplot2." https://ggplot2.tidyverse.org/articles/extending-ggplot2.html (accessed July 13, 2026).

[3] J. H. Larkin and H. A. Simon, "Why a Diagram Is (Sometimes) Worth Ten Thousand Words," *Cognitive Science*, vol. 11, no. 1, pp. 65–100, 1987, doi: 10.1111/j.1551-6708.1987.tb00863.x.

[4] G. Li, M. Tian, Q. Xu, M. J. McGuffin, and X. Yuan, "GoTree: A Grammar of Tree Visualizations," in *Proc. CHI*, 2020, pp. 1–13, doi: 10.1145/3313831.3376297.

[5] Z. Liu, C. Chen, and J. Hooker, "Manipulable Semantic Components: A Computational Representation of Data Visualization Scenes," *IEEE Transactions on Visualization and Computer Graphics*, 2024, doi: 10.1109/TVCG.2024.3456296.

[6] Z. Liu, C. Chen, F. Morales, and Y. Zhao, "Atlas: Grammar-Based Procedural Generation of Data Visualizations," in *IEEE Transactions on Visualization and Computer Graphics*, 2021, pp. 171–175, doi: 10.1109/VIS49827.2021.9623315.

[7] A. M. McNutt, "No Grammar to Rule Them All: A Survey of JSON-Style DSLs for Visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 1, pp. 160–170, 2022, doi: 10.1109/TVCG.2022.3209460.

[8] D. Park, S. M. Drucker, R. Fernandez, and N. Elmqvist, "ATOM: A Grammar for Unit Visualizations," *IEEE Trans. Visual Comput. Graphics*, vol. 24, no. 12, pp. 3032–3043, 2017, doi: 10.1109/TVCG.2017.2785807.

[9] S. Pinker, "A Theory of Graph Comprehension," *Artificial Intelligence and the Future of Testing*, vol. 73, p. 126, 1990, doi: 10.4324/9781315808178.

[10] J. Pollock, C. Mei, G. Huang, E. Evans, D. Jackson, and A. Satyanarayan, "Bluefish: Composing Diagrams with Declarative Relations," in *Proceedings of the ACM Symposium on User Interface Software and Technology*, 2024, pp. 1–21, doi: 10.1145/3654777.3676465.

[11] J. Pollock and A. Satyanarayan, "GoFish: A Grammar of More Graphics!," *IEEE Transactions on Visualization and Computer Graphics*, 2025.

[12] H. Shi, Y. Wang, J. Chen, C. Wang, and B. Lee, "PiCCL: Data-Driven Composition of Bespoke Pictorial Charts," *IEEE Trans. Visual Comput. Graphics*, 2025, doi: 10.1109/TVCG.2025.3634264.

[13] J. Wagemans *et al.*, "A Century of Gestalt Psychology in Visual Perception: I. Perceptual Grouping and Figure–Ground Organization," *Psychological Bulletin*, vol. 138, no. 6, pp. 1172–1217, 2012, doi: 10.1037/a0029333.

[14] L. Wilkinson, *The Grammar of Graphics*, 2nd ed. Springer, 2005.